

LoRa P2P RX Application

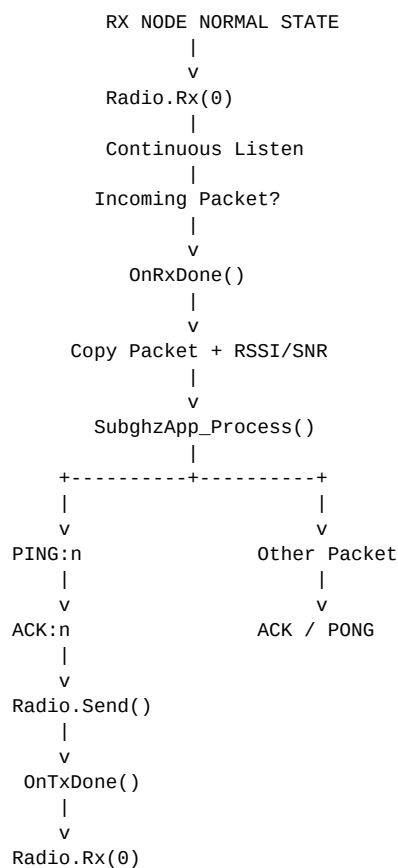
Detailed Technical Explanation of subghz_phy_app.c Receiver Code

Purpose: This document explains the LoRa Point-to-Point receiver application. The receiver continuously listens for incoming packets, measures RSSI and SNR, creates an ACK/PONG response, transmits that response, and immediately returns to continuous receive mode.

Role in the system: This code is designed to work with the previously documented LoRa P2P transmitter. The TX node sends packets such as PING:1, while this RX node receives them and responds with ACK:1 or PONG.

1. System Overview

The receiver implements a simple request-response protocol. Its normal state is continuous listening. When a packet arrives, the radio callback copies the payload and stores its signal information. The main application task then interprets the payload and sends an appropriate response.



2. Main Communication Cycle

Step	Receiver Action
1	Initialize the Sub-GHz radio and configure LoRa parameters.
2	Enter <code>Radio.Rx(0)</code> for continuous listening.
3	A LoRa packet arrives and <code>OnRxDone()</code> is called.
4	Copy the payload into Buffer and store RSSI and SNR.
5	Schedule <code>SubghzApp_Process()</code> using STM32 sequencer.

6	Inspect the received payload.
7	Generate ACK:n, PONG, or ACK.
8	Transmit the reply using Radio.Send().
9	After OnTxDone(), return immediately to Radio.Rx(0).

3. Application State Machine

```
typedef enum
{
    APP_IDLE,
    APP_RX,
    APP_RX_TIMEOUT,
    APP_RX_ERROR,
    APP_TX,
    APP_TX_TIMEOUT,
} AppStates_t;
```

State	Meaning
APP_IDLE	No pending event.
APP_RX	A valid packet has been received.
APP_RX_TIMEOUT	Receive operation timed out.
APP_RX_ERROR	Radio reported a receive error.
APP_TX	The ACK/PONG response was transmitted successfully.
APP_TX_TIMEOUT	The response transmission failed or timed out.

4. Important Variables

Variable	Purpose
State	Stores the current application event.
Buffer	Stores received data and is reused to transmit the reply.
BufferSize	Length of the current packet.
RssiValue	RSSI measured for the received packet.
SnrValue	SNR measured for the received packet.
RadioEvents	Contains callback function pointers for radio events.
huart1	UART used for debugging and status messages.

5. Initialization: SubghzApp_Init()

The initialization function first registers the application processing task with the STM32 sequencer.

```
UTIL_SEQ_RegTask(  
    SUBGHZ_PHY_PROCESS_TASK,  
    UTIL_SEQ_RFU,  
    SubghzApp_Process  
);
```

The radio callbacks are then assigned and the radio driver is initialized.

```
RadioEvents.TxDone      = OnTxDone;  
RadioEvents.RxDone      = OnRxDone;  
RadioEvents.TxTimeout   = OnTxTimeout;  
RadioEvents.RxTimeout   = OnRxTimeout;  
RadioEvents.RxError     = OnRxError;  
  
Radio.Init(&RadioEvents);
```

6. LoRa Radio Configuration

The receiver uses the same fundamental LoRa configuration parameters as the transmitter. Both devices must use compatible settings for successful communication.

```
Radio.SetChannel(RF_FREQUENCY);  
  
Radio.SetTxConfig(MODEM_LORA,  
    TX_OUTPUT_POWER,  
    0,  
    LORA_BANDWIDTH,  
    LORA_SPREADING_FACTOR,  
    LORA_CODINGRATE,  
    ...);  
  
Radio.SetRxConfig(MODEM_LORA,  
    LORA_BANDWIDTH,  
    LORA_SPREADING_FACTOR,  
    LORA_CODINGRATE,  
    ...);
```

Parameter	Why It Matters
RF Frequency	TX and RX must operate on compatible channels.
Bandwidth	Affects sensitivity, data rate, and airtime.
Spreading Factor	Higher SF generally improves sensitivity but increases airtime.
Coding Rate	Adds error correction at the cost of additional airtime.
Preamble Length	Allows the receiver to synchronize with the incoming signal.
CRC	Helps detect corrupted packets.
IQ Inversion	Must be configured compatibly between the communicating nodes.

7. Continuous Listening Mode

```
Radio.Rx(0);
```

The value 0 is used here as an indefinite/continuous receive operation in this application configuration. The receiver begins listening immediately after initialization and returns to listening after each response.

This creates a very simple receiver role: the radio spends most of its time waiting for a packet and only briefly switches to TX mode when it must send an acknowledgment.

8. OnRxDone(): Packet Reception Callback

```
BufferSize = size > BUFFER_SIZE ? BUFFER_SIZE : size;

memcpy(Buffer, payload, BufferSize);

RssiValue = rssi;
SnrValue = LoraSnr_FskCfo;

State = APP_RX;

UTIL_SEQ_SetTask(
    SUBGHZ_PHY_PROCESS_TASK,
    UTIL_SEQ_RFU
);
```

The callback performs four main operations: limits the received size, copies the payload, stores the signal measurements, and schedules the main processing task.

Keeping the callback short is good embedded-system design because radio callbacks may run in timing-sensitive contexts.

9. Safe String Termination

```
Buffer[
    BufferSize < BUFFER_SIZE
    ? BufferSize
    : BUFFER_SIZE - 1
] = '\0';
```

The received LoRa payload is raw binary data from the radio driver and is not guaranteed to contain a null terminator. Since this application treats packets as strings, it adds a terminating null character before using `strncmp()`, `snprintf()`, and UART logging.

10. Packet Parsing and Reply Logic

The receiver checks the beginning of the received packet and chooses a reply.

Case A: PING with Sequence Number

```
if (strncmp((char *)Buffer, "PING:", 5) == 0)
{
    snprintf(ackMsg,
             sizeof(ackMsg),
             "ACK:%s",
             (char *)Buffer + 5);
}
```

Example:

```
Received:  PING:25
Reply:     ACK:25
```

The expression `Buffer + 5` points to the characters immediately after `PING:`. Therefore, the receiver copies the sequence or remaining text into the ACK response.

Case B: HELLO or Generic PING

```
else if (
    strncmp((char *)Buffer, "HELLO", 5) == 0 ||
    strncmp((char *)Buffer, "PING", 4) == 0
)
{
    snprintf(ackMsg, sizeof(ackMsg), "PONG");
}
```

Examples:

```
Received: HELLO
Reply:    PONG
```

```
Received: PING
Reply:    PONG
```

Case C: Unknown Packet

```
else
{
    snprintf(ackMsg, sizeof(ackMsg), "ACK");
}
```

Any other packet receives a generic ACK. This provides a basic fallback response.

11. Reply Decision Table

Received Packet	Generated Reply
PING:1	ACK:1
PING:25	ACK:25
PING:HELLO	ACK:HELLO
HELLO	PONG
PING	PONG
Any other non-empty packet	ACK

12. RSSI and SNR Logging

```
[RX] Received: '%s'
(RSSI=%d dBm, SNR=%d dB)
-> Replying %s
```

The receiver records RSSI and SNR for the packet it received. These measurements describe the link from the TX node to this RX node.

RSSI indicates received signal strength. **SNR** indicates the relationship between the desired signal and the noise level. LoRa can successfully decode signals at low and sometimes negative SNR values depending on spreading factor and other configuration parameters.

13. Transmitting the ACK

```
uint16_t ackLen = (uint16_t)strlen(ackMsg);

memcpy(Buffer, ackMsg, ackLen);

BufferSize = ackLen;

Radio.Send(Buffer, BufferSize);
```

The reply string is copied into the shared radio buffer and transmitted. The same Buffer is reused for both reception and transmission.

14. APP_TX: Return to Receive Mode

```
case APP_TX:
{
    // ACK successfully transmitted

    Radio.Rx(0);
    break;
}
```

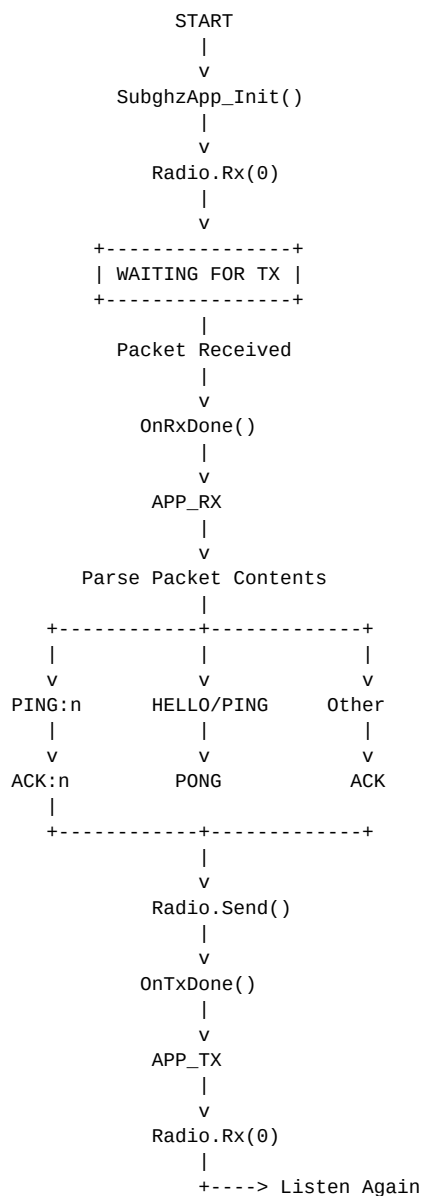
After the radio reports successful transmission, the receiver immediately returns to continuous listening. This is essential because the receiver must be ready for the next PING.

15. Timeout and Error Recovery

```
case APP_RX_TIMEOUT:
case APP_RX_ERROR:
case APP_TX_TIMEOUT:
{
    // Print diagnostic message
    Radio.Rx(0);
    break;
}
```

Any receive timeout, receive error, or transmit timeout causes the application to restart listening. This provides a simple self-recovery mechanism.

16. Complete Receiver State Flow



17. Example UART Output

```
--- RX Init Complete: Ready & Listening ---

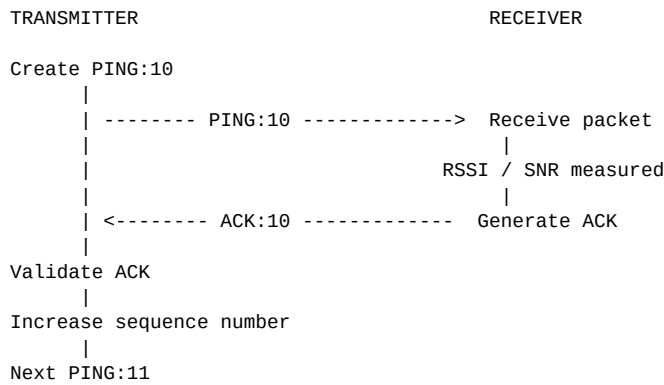
[RX] Received: 'PING:1'
(RSSI=-78 dBm, SNR=8 dB)
-> Replying ACK:1
[RX] ACK sent. Resuming continuous listen...
```

```
[RX] Received: 'PING:2'
(RSSI=-82 dBm, SNR=5 dB)
-> Replying ACK:2
```

```
[RX] ACK sent. Resuming continuous listen...
```

18. How This RX Works With the TX Application

The transmitter and receiver together implement the following application protocol.



19. Strengths of This Receiver Design

Feature	Benefit
Continuous listening	Receiver is normally ready for incoming packets.
Sequence-aware ACK	PING:n can receive ACK:n.
RSSI/SNR logging	Provides useful link diagnostics.
Automatic return to RX	Receiver resumes listening after every reply.
Fallback ACK	Unknown non-empty packets still receive a response.
Event-driven processing	Callbacks remain lightweight and processing is deferred.

20. Recommended Improvements

Improvement	Reason
Add explicit packet format	Use message type, sequence number, payload length, and optional checksum.
Validate exact sequence number	Useful for rejecting delayed or unrelated packets.
Avoid accepting every unknown packet	A generic ACK can hide protocol errors.
Add duplicate detection	Prevent responding repeatedly to the same retransmitted packet if required.
Use non-blocking UART logging	Blocking UART calls can affect timing in larger applications.
Add counters	Track received packets, CRC errors, timeouts, duplicates, and responses.
Protect shared buffer carefully	As the application grows, ensure RX/TX buffer reuse is synchronized.

21. Important Technical Notes

The code treats received payloads as text strings. If future applications need binary sensor data, the parsing and null-termination logic should be redesigned around packet lengths rather than string functions.

The RX and TX devices must use compatible radio configuration values. A mismatch in frequency, bandwidth, spreading factor, coding rate, CRC settings, or IQ inversion can prevent communication.

This receiver measures RSSI and SNR only for successfully delivered packets through OnRxDone(). A complete link analysis may additionally track timeouts, CRC errors, and packet loss.

22. Final Summary

This receiver is a continuously listening LoRa P2P node. Its primary operation is straightforward: listen, receive, measure link quality, generate an ACK/PONG response, transmit the response, and return to listening.

Together with the transmitter code, it forms a basic bidirectional heartbeat system capable of connection detection, sequence acknowledgment, automatic retry, and RSSI/SNR monitoring. It provides a good starting point for a self-healing LoRa P2P communication system.